Bitácora de Inteligencia Artificial - Examen Parcial 2

Alumno: Diego Baruc Zaid Materia: Programación Orientada a Objetos Práctica:

p115_SegundoExamenParcial

FASE 1: ANÁLISIS Y MODELADO UML (MERMAID)

Prompt enviado a la IA (Fase 1):

"Necesito modelar un sistema de gestión vehicular en Java. Las entidades principales son la superclase abstracta Vehiculo, dos clases derivadas llamadas CamionCarga y AutobusPasajeros, y la clase gestora Flota. Por favor, genera únicamente el diagrama de clases en formato Mermaid para previsualizarlo."

```
classDiagram
    class Vehiculo {
        <<abstract>>
        # String marcaUnit
        # String modeloUnit
        # int capCapacidad
        # double cuotaBase
        # double cuotaTotal
        + Vehiculo(String, String, int, double)
        + calcularBonoDriver()* double
    }
    class CamionCarga {
        - int numEjes
        - double kmRecorridos
        + calcularBonoDriver() double
    }
    class AutobusPasajeros {
        - int rutasAsignadas
        - int paxMovilizados
        + calcularBonoDriver() double
    }
    class Flota {
        - String nomDivision
        - List~Vehiculo~ listaUnidades
        + registrarUnidad(Vehiculo) void
        + reporte() void
    }
    Vehiculo <|-- CamionCarga
    Vehiculo <|-- AutobusPasajeros
    Flota "1" o-- "*" Vehiculo
```

FASE 2: ARQUITECTURA Y ESQUELETO DE CLASES BASE

Prompt enviado a la IA (Fase 2):

"Genera exclusivamente el código esqueleto en Java para las clases Vehiculo, CamionCarga y AutobusPasajeros. Define únicamente atributos privados/protegidos, constructores base y métodos getters/setters. No agregues aún métodos toString ni algoritmos de cálculo de bonificaciones."

```
// Vehiculo.java (Esqueleto de la superclase)
package p115_SegundoExamenParcial;

public abstract class Vehiculo {
    protected String marcaUnit;
    protected String modeloUnit;
    protected int capCapacidad;
    protected double cuotaBase;
    protected double cuotaTotal;
```


FASE 3: IMPLEMENTACIÓN POLIMÓRFICA Y REGLAS DE NEGOCIO

Prompt enviado a la IA (Fase 3):

"Implementa la firma del método abstracto calcularBonoDriver() en Vehiculo y añade las implementaciones sobreescritas con @Override en CamionCarga y AutobusPasajeros según las fórmulas de bonificación y costos. Asegúrate de invocar super() en los constructores."

```
// CamionCarga.java
package p115_SegundoExamenParcial;

public class CamionCarga extends Vehiculo {
    private int numEjes;
    private double kmRecorridos;

    public CamionCarga(String marca, String modelo, int capacidad, double costoBase, int ejes, double kilometraje) {
        super(marca, modelo, capacidad, costoBase);
        this.numEjes = ejes;
        this.kmRecorridos = kilometraje;
        this.setCuotaTotal(this.getCuotaBase() + this.calcularBonoDriver());
    }

    @Override
    public double calcularBonoDriver() {
        return (getCuotaBase() * 0.10) + (numEjes * 50.0) + (kmRecorridos * 0.5);
    }

    @Override
    public String toString() {
        return "CamionCarga [" + super.toString() + ", Ejes=" + numEjes + ", Kilometraje=" + kmRecorridos +
                                                ", Bono=" + calcularBonoDriver() + ", CostoTotal=" + getCuotaTotal() + "]";
    }
}


// AutobusPasajeros.java
package p115_SegundoExamenParcial;

public class AutobusPasajeros extends Vehiculo {
    private int rutasAsignadas;
    private int paxMovilizados;

    public AutobusPasajeros(String marca, String modelo, int capacidad, double costoBase, int rutas, int pasajeros) {
        super(marca, modelo, capacidad, costoBase);
        this.rutasAsignadas = rutas;
        this.paxMovilizados = pasajeros;
        this.setCuotaTotal(this.getCuotaBase() + this.calcularBonoDriver());
    }

    @Override
    public double calcularBonoDriver() {
        return (getCuotaBase() * 0.15) + (rutasAsignadas * 120.0) + (paxMovilizados * 0.1); }

    @Override
    public String toString() {
        return "AutobusPasajeros [" + super.toString() + ", Rutas=" + rutasAsignadas + ", Pasajeros
                Transportados=" + paxMovilizados +
                                                ", Bono=" + calcularBonoDriver() + ", CostoTotal=" + getCuotaTotal() + "]";
    }
}
```

FASE 4: GESTOR DE COLECCIÓN HETEROGÉNEA (FLOTA.JAVA) Bitácora de IA -


```
// Flota.java
package p115_SegundoExamenParcial;
import java.util.ArrayList;
import java.util.List;

public class Flota {
    private String nomDivision;
    private String ciudadBase;
```

```
    private List<Vehiculo> listaUnidades;

    public Flota(String nombre, String ciudad) {
        this.nomDivision = nombre;
        this.ciudadBase = ciudad;
        this.listaUnidades = new ArrayList<>();
    }

    public void registrarUnidad(Vehiculo unidad) { this.listaUnidades.add(unidad); } public void
    agregarVehiculo(Vehiculo vehiculo) { this.registrarUnidad(vehiculo); }

    public double getTotalBono() {
        double acumulado = 0.0;
        for (Vehiculo elem : listaUnidades) acumulado += elem.calcularBonoDriver();
        return acumulado;
    }

    public double getTotalCosto() {
        double acumulado = 0.0;
        for (Vehiculo elem : listaUnidades) acumulado += elem.getCuotaTotal();
        return acumulado;
    }

    public void reporte() {
        System.out.println(">> Reporte Detallado de la Flota <<");
        for (Vehiculo elem : listaUnidades) System.out.println(elem.toString());
    }
}
```

FASE 5: EJECUCIÓN Y EXPLICACIÓN DE LATE BINDING

Prompt enviado a la IA (Fase 5):

"Escribe la clase App con el método main con los datos de prueba del examen y redacta un bloque de comentarios que explique técnicamente cómo funciona el polimorfismo y el Late Binding (enlace tardío)."

```
// App.java
package p115_SegundoExamenParcial;

public class App {
    public static void main(String[] args) {
        Flota flotaNorte = new Flota("Norte", "Monterrey");

        flotaNorte.registrarUnidad(new CamionCarga("Volvo", "FMX", 30000, 2500.0, 4, 5000.0)); flotaNorte.registrarUnidad(new
        CamionCarga("Kenworth", "T680", 25000, 1800.0, 3, 4200.0)); flotaNorte.registrarUnidad(new
        AutobusPasajeros("Mercedes", "O500", 50, 1200.0, 5, 8500)); flotaNorte.registrarUnidad(new AutobusPasajeros("Irizar",
        "i6", 40, 1500.0, 8, 12000)); flotaNorte.registrarUnidad(new AutobusPasajeros("Scania", "K360", 45, 950.0, 4, 6200));

        flotaNorte.reporte();
        System.out.println(flotaNorte);
    }
}
```

Análisis Técnico de Polimorfismo y Late Binding (Enlace Tardío):

En este programa, la colección listaUnidades almacena objetos de subclases distintas (CamionCarga y AutobusPasajeros) agrupados bajo el tipo general Vehiculo. Al invocar el método calcularBonoDriver() dentro de las iteraciones de la clase Flota, el compilador solo valida la existencia de la firma en la superclase abstracta.